

CampusIQ

Smart College Management & Analytics Platform

Technical Documentation

Version 1.0.0

September 2026

Table of Contents

- 1. Executive Summary2
- 2. Project Objectives3
- 3. User Roles and RBAC4
- 4. System Architecture5
- 5. Technology Stack6
- 6. Repository Structure7
- 7. Authentication and Security8
- 8. Module Documentation9
- 9. API Documentation10
- 10. Database Design11
- 11. Frontend Architecture12
- 12. Backend Architecture13
- 13. Docker & Deployment14
- 14. Installation & Setup15
- 15. Testing & Quality Assurance16
- 16. Demo Accounts17
- 17. Screenshots18
- 18. Known Limitations19
- 19. Future Scope20
- 20. License21
- 21. Changelog22

1. Executive Summary

CampusIQ is a comprehensive web-based Smart College Management and Analytics Platform designed to streamline academic operations, enhance student-faculty engagement, and provide data-driven insights for institutional decision-making.

Problem: Educational institutions face challenges in managing student attendance, tracking assignments, generating analytics, and maintaining secure access control.

Solution: Unified platform with real-time attendance tracking, assignment management, automated grading, role-based dashboards, and exportable reports.

2. Project Objectives

CampusIQ achieves the following objectives:

- Attendance Management: Real-time tracking with predictive analytics
- Assignment Workflows: End-to-end lifecycle from creation to grading
- Data-Driven Decisions: Comprehensive reports for all stakeholders
- Secure Access Control: 5-role RBAC system
- Department Oversight: HOD-specific dashboards
- Academic Planning: Timetable management
- Communication: Priority-based announcements
- Project Management: Group tracking with milestones

3. User Roles and RBAC

CampusIQ implements a 5-role system:

- ADMIN: Full system access, user management, audit logs
- HOD: Department analytics, student/faculty management
- FACULTY: Attendance marking, assignment management, grading
- CR: Class attendance view, feedback, requests
- STUDENT: Personal attendance, assignments, marks

4. System Architecture

Layered architecture with three main components:

Frontend Layer: React 18 with TypeScript, component-based UI, role-based routing

Backend Layer: FastAPI with Pydantic validation, SQLAlchemy ORM, Alembic migrations

Database Layer: SQLite for development, PostgreSQL for production

5. Technology Stack

Frontend: React 18, TypeScript 5, Vite 5, Tailwind CSS 3, React Router 6, TanStack Query 5

Backend: FastAPI 0.141, Python 3.12+, SQLAlchemy 2.0, Alembic 1.20, PyJWT 2.14, bcrypt 5.0

Database: SQLite (dev), PostgreSQL (prod)

Security: JWT (30 min access + 7 day refresh), bcrypt hashing, RBAC on every endpoint

6. Repository Structure

CampusIQ/

backend/
 app/api/v1/ (API endpoints)
 app/core/ (config, security)
 app/models/ (database models)
 app/services/ (business logic)
 tests/ (test suite)
 alembic/ (migrations)
frontend/
 src/ (React components)
docs/
.env.example
LICENSE
README.md
docker-compose.yml

7. Authentication and Security

Authentication Flow:

1. Login generates JWT access + refresh tokens
2. Access token in Authorization header
3. Token refresh with refresh token
4. Logout revokes refresh tokens

Password: bcrypt hashed, minimum 6 chars

Authorization: RBAC on every endpoint, resource-level scoping

Audit: All security actions logged

8. Module Documentation

Attendance: Track and analyze with zone classification

Assignments: Create, submit, grade with deadlines

Marks: Performance tracking and calculations

Projects: Group work with milestone tracking

Announcements: Broadcast with priority levels

Reports: CSV/PDF export for all modules

Audit Logs: Admin-only activity tracking

9. API Documentation

Base: /api/v1 | Docs: /docs

POST /auth/login - Authenticate

GET /attendance - List records

POST /assignments - Create assignment

GET /reports/attendance - Generate report

GET /audit-logs - View audit trail

10. Database Design

Core Entities:

- User: id, email, role, status
- Student: user_id, enrollment, department, section
- Faculty: user_id, department, designation
- Assignment: title, subject, faculty, deadline
- Attendance: student_id, date, status

Database: SQLite (dev), PostgreSQL (prod)

11. Frontend Architecture

React 18 + TypeScript
Component-based UI with role-based routing
React Query for data fetching
Protected routes per user role
Tailwind CSS styling

12. Backend Architecture

FastAPI application structure:
API Routes - Dependencies/RBAC - Services - Database
Pydantic for request validation
SQLAlchemy ORM
Alembic migrations
Service layer for business logic

13. Docker & Deployment

Services:
- db: postgres:16-alpine (5432)
- backend: FastAPI app (8000)
- frontend: React app (5173)

Environment: DATABASE_URL, JWT_SECRET, CORS_ORIGINS

14. Installation & Setup

Backend:
pip install -r requirements.txt
alembic upgrade head
uvicorn app.main:app --reload

Frontend:
npm install
npm run dev

15. Testing & Quality Assurance

Backend: pytest (114 tests passing)
Frontend: npm run build, npx tsc --noEmit, npm run lint

16. Demo Accounts

DEVELOPMENT ONLY - Change before production:

- admin@campusiq.edu / Admin@123 (ADMIN)
- hod.cse@campusiq.edu / Hod@12345 (HOD)
- faculty@campusiq.edu / Faculty@123 (FACULTY)
- cr@campusiq.edu / Cr@12345 (CR)
- adnan@campusiq.edu / Student@123 (STUDENT)

17. Screenshots

Dashboard and workflow screenshots to be added.

Include:

- Admin dashboard
- HOD analytics
- Faculty assignment management
- Student portal

18. Known Limitations

- Rate limiting defaults need adjustment for high traffic
- AI integration requires external provider
- 10MB file upload limit
- SQLite not recommended for production

19. Future Scope

Potential enhancements:

- WebSocket real-time notifications
- Multi-factor authentication
- Mobile application
- SIS/LMS integration
- Multi-language support

20. License

MIT License

21. Changelog

v1.0.0 - Initial release:

- 5-role RBAC system
- Full attendance management
- Assignment workflow
- Marks and performance tracking
- Report generation (CSV/PDF)
- Audit logging
- Docker deployment support